Arabic Policy Analyzer

Multi-Document Agentic RAG System

Detailed Project Report

System Type	Multi-document Agentic Retrieval-Augmented Generation system
Document Scope	Arabic and English policy and regulation PDFs
Current Collection	4 documents, 105 pages, 152 retrievable chunks
Core Retrieval Method	Hybrid retrieval using FAISS semantic search and BM25 keyword search
Generation Model	gpt-4o-mini with source-grounded answer generation

Table of Contents

1. Introduction
2. Project Objective
3. Data Sources and Indexed Document Collection
4. System Architecture
5. Knowledge Base Ingestion
6. Chunking and Indexing Configuration
7. Purpose of the Main Project Files
8. AI Tools and Technologies Used
9. Why Both FAISS and chunks.json Are Used
10. Question Routing and Task Detection
11. Hybrid Retrieval Approach
12. Cross-Encoder Re-ranking
13. Grounded Answer Generation
14. Evaluation Setup
15. Evaluation Results
16. Key Design Decisions
17. Risks and Improvement Opportunities
18. Conclusion

1. Introduction

The Arabic Policy Analyzer is a multi-document question-answering system developed to support users in searching, understanding, and analyzing policy and regulation documents written in Arabic and English. The system is based on an Agentic Retrieval-Augmented Generation architecture, where user questions are routed to the most relevant document domain, relevant evidence is retrieved from indexed documents, candidate passages are re-ranked, and a grounded answer is generated using a large language model.

The project was designed for policy and regulatory documents, which are often long, formal, and difficult to search manually. These documents may contain articles, clauses, definitions, controls, exceptions, and procedures, where the meaning of one statement can depend on surrounding sections. Because of this structure, a simple keyword search or a basic chatbot may not be enough to provide reliable answers.

The Arabic Policy Analyzer addresses this challenge by building a searchable knowledge base from multiple policy PDFs. When a user asks a question, the system does not rely on the language model's general knowledge. Instead, it retrieves relevant text chunks from the indexed documents and uses only those chunks to generate the final answer. This makes the system more suitable for policy-focused use cases, where traceability and source grounding are essential.

The current implementation covers four main regulatory domains: data sharing, cybersecurity controls, critical systems cybersecurity, and personal data protection. The document collection includes both Arabic and English files, which allows the system to support bilingual policy analysis.

2. Project Objective

The main objective of this project is to build a reliable assistant that can answer questions from multiple policy and regulation PDFs. Instead of requiring users to manually open and search each document, the system provides a single interface where users can ask natural-language questions and receive answers supported by retrieved source content.

The project aims to achieve the following objectives:

1. Support Arabic and English policy documents.
2. Handle multiple regulatory domains in one searchable knowledge base.
3. Route user questions to the most relevant document domain or document.
4. Improve retrieval quality by combining semantic search and keyword search.
5. Use re-ranking to select the strongest source passages before generation.
6. Generate answers that are grounded in retrieved evidence.
7. Display simplified source information to help users trace the answer back to the original documents.
8. Reduce the risk of hallucination by instructing the language model to answer only from retrieved sources.

The system is therefore not designed as a general-purpose chatbot. It is designed as a document-grounded policy assistant that helps users retrieve and understand information from official policy and regulation documents.

3. Data Sources and Indexed Document Collection

The current version of the Arabic Policy Analyzer was built using four policy and regulation PDF documents. These documents cover different but related domains, including data sharing, cybersecurity controls, critical systems cybersecurity, and personal data protection. The collection includes three Arabic documents and one English document.

The indexed collection contains a total of 105 pages and was divided into 152 retrievable chunks during the ingestion process. Each document was assigned metadata such as document ID, file name, title, domain, domain label,

language, authority, document type, source path, page count, and chunk count. This metadata is used by the system for routing, filtering, source tracking, and displaying document information in the Streamlit sidebar.

File Name	Domain	Language	Authority	Pages	Chunks
CriticalSystemsCybersecurityControls.pdf	Critical Systems Cybersecurity	Arabic	NCA	32	37
DataSharingPolicyAR.pdf	Data Sharing	Arabic	SDAIA	17	19
EssentialCybersecurityControls-NCA.pdf	Cybersecurity Controls	Arabic	NCA	40	67
PersonalDataProtectionLaw-English.pdf	Personal Data Protection	English	SDAIA	16	29
Total				**105**	**152**

Table 1. Indexed policy and regulation document collection.

The document collection reflects a realistic use case for policy analysis. The documents come from more than one authority and cover related regulatory areas. For example, a question about sharing personal data may require understanding both data sharing requirements and personal data protection obligations. This is one reason the system was designed as a multi-document RAG system rather than a single-document question-answering tool.

The documents also present several technical challenges. Arabic PDFs may include right-to-left formatting issues, inconsistent character forms, or extraction errors. Some files include repeated page headers, footers, page numbers, classification labels, and authority names. These elements can negatively affect retrieval quality if they are treated as normal document content. Therefore, preprocessing and metadata enrichment are important parts of the system design.

4. System Architecture

The Arabic Policy Analyzer follows a four-stage Agentic RAG pipeline: knowledge base ingestion, indexing, hybrid retrieval and re-ranking, and grounded answer generation.

The overall flow can be summarized as follows:

```
User Question
    ↓
Streamlit Interface
    ↓
RAG Agent
    ↓
Question Routing
    ↓
Hybrid Retrieval: FAISS + BM25
    ↓
Candidate Merging
    ↓
Cross-Encoder Re-ranking
    ↓
Grounded Answer Generation
    ↓
Answer + Sources Displayed to User
```

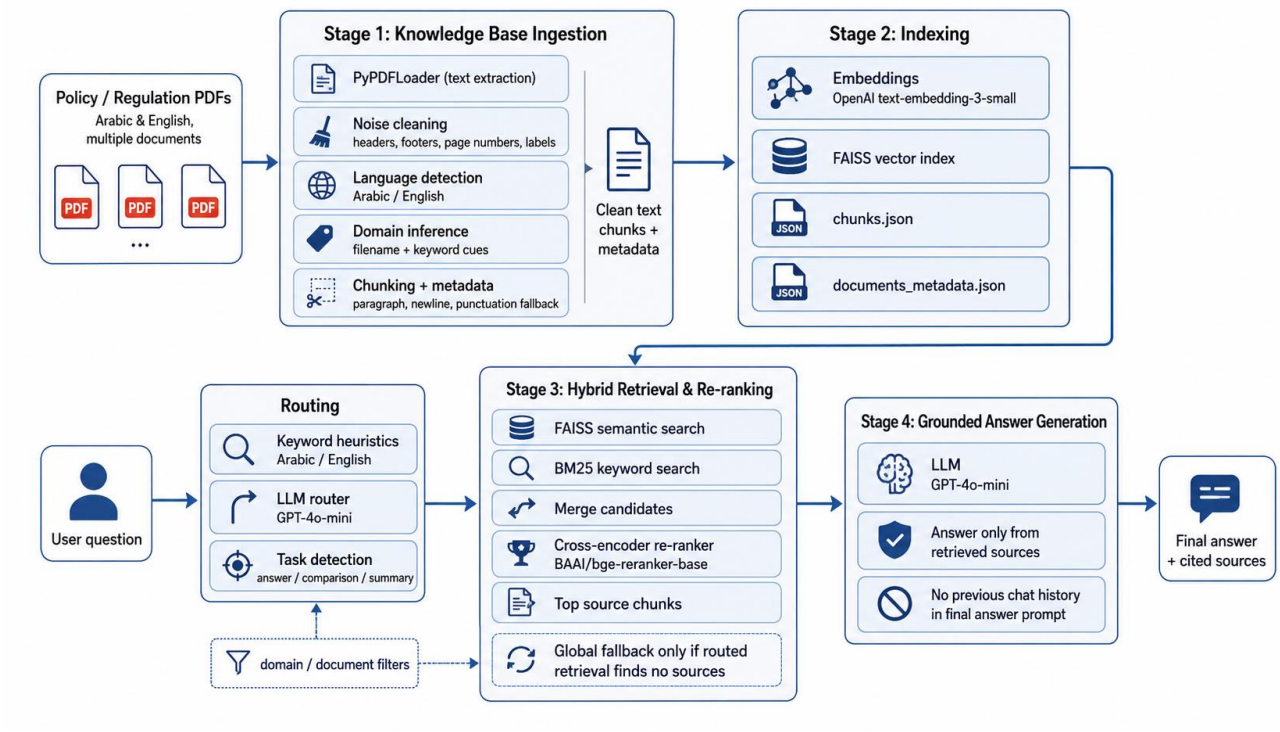

Fig. 1. Architecture of the Arabic Policy Analyzer.

The first stage prepares the documents and builds the knowledge base. The second stage creates the vector index and supporting metadata files. The third stage retrieves and re-ranks evidence based on the user question. The fourth stage generates an answer using the selected source chunks.

This architecture was selected because policy and regulation documents require both semantic understanding and precise source grounding. The system must be able to retrieve passages that are conceptually relevant while also preserving exact policy terms, document domains, and source references.

5. Knowledge Base Ingestion

The ingestion stage is handled by ingest.py. This stage reads all PDF files stored in the documents/ folder and prepares them for retrieval.

The system uses PyPDFLoader to extract text from each PDF. After extraction, the text is cleaned to remove repeated noise that commonly appears in policy and government documents. This includes headers, footers, page numbers, classification labels, and authority names. Removing these repeated elements improves retrieval quality because the index becomes more focused on the actual policy content.

After cleaning, the system detects whether the document is Arabic or English. It also infers the document domain using information from the file name and document content. For example, a file related to cybersecurity controls is assigned to the cybersecurity domain, while a document related to personal data protection is assigned to the personal data protection domain.

Each extracted text segment is then prepared for chunking. The goal is to produce text chunks that are small enough for retrieval and answer generation, but large enough to preserve the meaning of policy clauses, definitions, requirements, and exceptions.

6. Chunking and Indexing Configuration

Chunking is a critical part of the system because retrieval quality depends heavily on how the original document content is divided. If chunks are too short, the system may lose the surrounding context needed to understand a policy rule or exception. If chunks are too long, retrieval may become less precise because a single chunk may contain several unrelated ideas.

The current implementation uses RecursiveCharacterTextSplitter with a generic separator strategy:

```
RecursiveCharacterTextSplitter(
    separators=GENERIC_SEPARATORS,
    chunk_size=1800,
    chunk_overlap=250,
    keep_separator=True,
)
```

This means that each chunk has a maximum size of approximately 1,800 characters, with an overlap of 250 characters between consecutive chunks. The overlap helps preserve continuity when a definition, article, requirement, or exception continues across a chunk boundary. The keep_separator=True setting keeps separators in the chunk text, which is useful for policy documents because punctuation and structural markers can help preserve meaning.

The splitter first attempts to divide text using larger natural boundaries, such as paragraphs and newlines. If these are not available or not sufficient, it falls back to smaller separators such as punctuation marks, commas, spaces, and finally character-level splitting. This flexible approach is useful for mixed Arabic and English PDFs because document formatting and extraction quality can differ across files.

In the current indexed collection, the four documents were split into 152 retrievable chunks across 105 pages. The chunk distribution differs across documents because each PDF has a different length, structure, and density of content.

After chunking, each chunk is stored with metadata, including document ID, file name, title, domain, domain label, language, authority, page number, chunk ID, and chunk index. This metadata supports routing, filtering, source tracking, debugging, and source display in the user interface.

After chunking and metadata enrichment, each chunk is embedded using OpenAI's text-embedding-3-small model. The resulting embeddings are stored in a FAISS vector index under indexes/policy_index/. At the same time, readable chunk text and metadata are saved in indexes/policy_index/chunks.json, while document-level metadata is saved in documents_metadata.json.

This design allows the system to support both semantic retrieval and keyword-based retrieval. FAISS stores the vector index for semantic search, while chunks.json stores the readable evidence and metadata needed for BM25 search, filtering, answer generation, and source tracing.

7. Purpose of the Main Project Files

7.1 ingest.py

The ingest.py file is responsible for building the knowledge base. It reads PDF files from the documents/ folder, extracts their text, cleans repeated noise, detects language, infers document domains, splits the content into chunks, and attaches metadata to each chunk.

It then creates embeddings using OpenAI's text-embedding-3-small model and saves the FAISS vector index. It also saves readable chunks in chunks.json and document-level metadata in documents_metadata.json. In simple terms, ingest.py converts raw policy PDFs into a searchable knowledge base.

7.2 rag_agent.py

The rag_agent.py file contains the main RAG agent logic. It loads the FAISS index, loads chunks.json as LangChain Document objects, loads documents_metadata.json, and builds a BM25 keyword retriever from the chunk text.

When a user submits a question, this file handles routing, retrieval, candidate merging, re-ranking, and answer generation. It identifies the likely domain or document, retrieves relevant chunks using FAISS and BM25, re-ranks the candidate chunks using a cross-encoder model, and sends the selected context to the LLM. This file is the main intelligence layer of the system.

7.3 second-app.py

The second-app.py file provides the Streamlit user interface. It loads the OpenAI API key from Streamlit secrets or environment variables, checks whether the knowledge base exists, displays indexed documents in the sidebar, and provides buttons for rebuilding the knowledge base and starting a new conversation.

The Streamlit app sends each user question to MultiDocumentRAGAgent.answer(). It then displays the generated answer and simplified source information, including the document domain and file name. This file is responsible for the user experience, while the retrieval and generation logic is handled by rag_agent.py.

8. AI Tools and Technologies Used

The Arabic Policy Analyzer uses a set of modern AI, NLP, and retrieval technologies. The system is implemented in Python and uses LangChain components to support document loading, chunk handling, and retrieval workflows.

PyPDFLoader is used to extract text from PDF documents. Custom preprocessing is then applied to clean repeated document noise and prepare the text for indexing.

OpenAI's text-embedding-3-small model is used to convert text chunks into dense vector representations. These embeddings allow the system to retrieve passages that are semantically related to the user question, even when the wording differs from the document text.

FAISS is used as the vector database for semantic retrieval. It stores the embeddings and allows the system to search for chunks based on vector similarity.

BM25 is used for keyword-based retrieval. This is important for policy documents because some questions depend on exact terms, article names, definitions, regulatory expressions, or Arabic legal phrases.

The system uses BAAI/bge-reranker-base as a cross-encoder re-ranker. After the initial retrieval step, the re-ranker compares each candidate chunk directly with the user question and assigns a relevance score. This improves the quality of the final context passed to the LLM.

gpt-4o-mini is used for routing, task detection, and answer generation. It helps classify the user question, identify whether the task is a normal answer, comparison, or summary, and generate the final response from the selected source chunks.

Streamlit is used to provide the user interface. It gives users a simple chat-based experience, displays indexed documents, and allows the knowledge base to be rebuilt when documents are added, removed, or updated.

9. Why Both FAISS and chunks.json Are Used

The system uses both FAISS and chunks.json because they serve different purposes.

FAISS stores the vector embeddings and supports fast semantic search. It helps the system retrieve chunks that are similar in meaning to the user question. For example, if a user asks about obligations related to personal data, FAISS can retrieve passages about privacy rights, processing requirements, or data controller responsibilities even if the exact wording is different.

However, FAISS alone is not enough. The LLM needs readable text to generate an answer, and the system also needs metadata for filtering, citation, debugging, and source display. This is why the readable text chunks and metadata are stored separately in chunks.json.

chunks.json is used for BM25 keyword retrieval, answer context construction, source metadata, filtering by domain or document, debugging and inspection, and displaying sources in the user interface.

```
FAISS = semantic search over vector embeddings
chunks.json = readable text and metadata
```

Together, they allow the system to retrieve relevant information and then use that information in a transparent and traceable way.

10. Question Routing and Task Detection

Question routing is one of the main agentic components of the system. Instead of searching all documents without direction, the system first attempts to understand the user question and identify the most relevant domain or document.

The router uses Arabic and English keyword heuristics supported by an LLM router using gpt-4o-mini. Keyword heuristics provide a fast way to detect obvious domain signals. For example, questions containing terms related to data sharing may be routed to the data sharing domain, while questions about cybersecurity controls may be routed to the cybersecurity domain.

If the question is less direct or uses more flexible wording, the LLM router helps classify the question more intelligently. This is useful when the user asks about a concept without using the exact document terminology.

The router also detects the task type. The current task categories include normal answer, comparison, and summary. This helps the system decide how to retrieve information and how to structure the final answer. For example, a comparison question may require evidence from more than one document or domain, while a summary question may require broader retrieval from a specific document.

Routing improves retrieval efficiency because the system can focus on the most relevant part of the knowledge base instead of treating all documents equally.

11. Hybrid Retrieval Approach

The retrieval stage combines FAISS semantic search and BM25 keyword search. This hybrid approach is important because policy and regulation documents require both meaning-based and exact-term retrieval.

FAISS is useful when the user's question is phrased differently from the document text. For example, a user may ask, "What are the rules for sharing data?" while the document may use formal terms such as "data sharing requirements" or "data exchange controls." Semantic search can still retrieve relevant passages because it searches based on meaning.

BM25 is useful when exact terms matter. In policy documents, terms such as Article, Clause, Personal Data, Critical Systems, Classification, or their Arabic equivalents may carry specific legal or regulatory meaning. BM25 helps ensure that exact terminology is not missed.

The system retrieves candidates from both methods, merges the results, removes duplicates, and applies domain or document filters when routing information is available. If routed retrieval does not return any useful sources, the system can fall back to a broader search across all indexed documents.

This retrieval strategy improves coverage because it combines the strengths of dense semantic retrieval and sparse keyword retrieval.

12. Cross-Encoder Re-ranking

After hybrid retrieval, the merged candidate chunks are passed to a cross-encoder re-ranker using BAAI/bge-reranker-base.

The initial retrieval stage is mainly used to collect potentially relevant chunks. However, not all retrieved chunks are equally useful. Some may be loosely related, while others may directly answer the user's question. The re-ranker improves precision by comparing each candidate chunk directly with the user question and assigning a relevance score.

The highest-scoring chunks are selected as the final evidence for answer generation. This helps ensure that the language model receives focused and relevant context rather than a large group of weakly related passages.

This step is particularly important for policy documents because small differences in wording can change the meaning of a requirement, exception, or control. Re-ranking helps the system select passages that are more closely aligned with the user's intent.

13. Grounded Answer Generation

After re-ranking, the selected source chunks are sent to gpt-4o-mini for answer generation. The model receives the user question and the selected evidence. It is instructed to answer only from the retrieved sources.

This design reduces the risk of hallucination. If the retrieved context does not contain enough information to answer the question, the model is expected to state that the available sources are insufficient instead of generating an unsupported answer.

The final response is displayed in the Streamlit interface with simplified source information. The sources currently include the document domain and file name. This gives the user a clear indication of which documents were used to generate the answer.

A key design decision is that previous chat history is not included in the final answer prompt. Although the interface provides a chat experience, each answer is generated from the current question and the retrieved source chunks only. This prevents the model from repeating an old answer or mixing previous conversation context with a new question.

14. Evaluation Setup

The system was evaluated by comparing the proposed Agentic RAG pipeline against a Simple-RAG baseline. The goal of this evaluation was to measure whether the additional components, such as routing, hybrid retrieval, and cross-encoder re-ranking, improved answer quality.

The Simple-RAG baseline represents a more direct retrieval process. It mainly retrieves chunks from the vector index and sends them to the language model for answer generation. This approach can work for simple questions, but it may struggle with policy documents because it does not include domain routing, keyword retrieval, or re-ranking.

The proposed Agentic RAG pipeline includes several improvements. It routes the question to the relevant domain or document, combines FAISS semantic retrieval with BM25 keyword retrieval, merges the retrieved candidates, re-ranks them using a cross-encoder model, and sends only the strongest chunks to the LLM.

The evaluation was conducted using test questions derived from the indexed policy documents. The RAGAS framework was used to assess answer quality. The main evaluation dimensions included groundedness and correctness.

Groundedness measures whether the generated answer is supported by retrieved source passages. Correctness measures whether the answer accurately reflects the expected information from the policy documents.

15. Evaluation Results

The evaluation results show that the proposed Agentic RAG pipeline improved answer quality compared with the Simple-RAG baseline.

Metric	Simple-RAG Baseline	Proposed Agentic RAG
Groundedness	3.0	8.7
Correctness	3.3	5.3

Table 2. Evaluation results comparing the Simple-RAG baseline with the proposed Agentic RAG system.

The strongest improvement was observed in groundedness. The baseline system achieved a groundedness score of 3.0, while the proposed system achieved 8.7. This suggests that the proposed pipeline was much better at generating answers supported by retrieved source content.

Correctness also improved. The baseline achieved a correctness score of 3.3, while the proposed system achieved 5.3. Although the improvement in correctness was smaller than the improvement in groundedness, it still shows that better retrieval and evidence selection contributed to more accurate answers.

These results indicate that the added components improved the reliability of the system. Routing helps narrow the search space to the most relevant document domain. Hybrid retrieval improves coverage by combining semantic similarity with exact keyword matching. Re-ranking improves precision by selecting the chunks that are most directly related to the user question.

For policy and regulation documents, these improvements are important because a response must not only sound fluent; it must also be supported by official source material.

16. Key Design Decisions

Several design decisions were made to improve the reliability and usability of the system.

- The system uses hybrid retrieval instead of relying only on vector search. This is important because policy documents often contain exact terms and regulatory phrases that may not be captured by semantic similarity alone.
- The system uses a cross-encoder re-ranker before answer generation. This improves the quality of the final context and reduces the chance of sending weak or unrelated evidence to the LLM.
- The system stores both FAISS vectors and readable chunks. FAISS supports semantic retrieval, while chunks.json stores the original text and metadata needed for BM25, citations, filtering, and answer generation.
- Previous chat history is excluded from the final answer prompt. This prevents the model from relying on older conversation turns and keeps each response grounded in the current question and retrieved evidence.
- The system supports multiple domains rather than a single document. This makes it more useful for realistic policy analysis, where questions may involve more than one regulatory area.

18. Conclusion

The Arabic Policy Analyzer is a multi-document Agentic RAG system designed to support policy and regulation analysis across Arabic and English documents. It combines document preprocessing, metadata enrichment, domain routing, hybrid retrieval, cross-encoder re-ranking, and grounded LLM-based answer generation.

The current knowledge base contains four official policy and regulation PDFs from SDAIA and NCA, covering data sharing, cybersecurity controls, critical systems cybersecurity, and personal data protection. Together, these

documents include 105 pages and were divided into 152 retrievable chunks using a chunk size of 1,800 characters and an overlap of 250 characters.

The evaluation results show that the proposed Agentic RAG pipeline outperformed the Simple-RAG baseline. Groundedness improved from 3.0 to 8.7, while correctness improved from 3.3 to 5.3. These results suggest that routing, hybrid retrieval, and cross-encoder re-ranking play an important role in improving the quality and reliability of generated answers.

Overall, the project provides a practical foundation for building a reliable policy assistant. It allows users to interact with complex regulatory documents through natural-language questions while keeping answers connected to the indexed source material. Future improvements may include more detailed citations, multi-domain routing, stronger Arabic normalization, automatic document-change detection, and broader evaluation across a larger question set.